Lecture 31: Fixed-Point DSP Implementation, Quantization Effects & Real-Time Embedded Signal Processing

Course: EE3621 — Digital Signal Processing

Level: Advanced undergraduate / industry-oriented extension

Suggested duration: 60–75 minutes

Prerequisites: Quantization, Z-transform, FIR/IIR filters, filter realization, frequency response

Learning mode: Theory + worked numerical examples + implementation case study

1. Why this lecture matters

The transfer function of a digital filter describes its ideal mathematical behaviour. A real embedded processor, however, stores numbers with a finite number of bits and executes a finite number of operations per second.

Consequently, an implementation must address:

- finite word length and number representation;
- coefficient and signal quantization;
- rounding and truncation;
- overflow, saturation and wrap-around;
- accumulator sizing;
- quantization noise and limit-cycle oscillations;
- memory, latency and real-time deadlines.

These are practical design concerns in motor drives, power-quality meters, protection relays, audio systems, biomedical instruments, communication equipment and industrial control.

Central engineering question: How closely will the implemented filter behave like the ideal filter, and can it finish every computation before the next sample arrives?

2. Fixed-point number representation

A fixed-point number uses a fixed binary-point position. A common convention is Qm.n, where n is the number of fractional bits. The exact meaning of `m` varies between tools, so always state the total word length and fractional-bit count explicitly.

For a signed 16-bit two's-complement value with 15 fractional bits (often written Q1.15):

- total bits: 16;
- fractional bits: 15;
- scale factor: 2^15 = 32768;
- representable range: -1 to 1 - 2^-15;
- resolution (one LSB): 2^-15 = 0.0000305176.

The quantized integer code for a real value x is approximately:

```
X = round(x × 2^15)
```

and the represented value is:

```
x_hat = X / 2^15.
```

Example 1 — Converting a decimal value to Q1.15

Represent x = 0.625 in signed Q1.15.

```
X = round(0.625 × 32768) = 20480
```

Binary representation: `0101 0000 0000 0000`

Decoding:

```
x_hat = 20480 / 32768 = 0.625
```

This value is represented exactly because 0.625 has a finite binary representation.

Important range warning

Q1.15 cannot represent +1.0. Its maximum positive value is:

```
(32767 / 32768) = 0.9999694824
```

A signal exceeding the representable range must be scaled, clipped or represented with more integer bits.

3. Quantization: rounding versus truncation

Quantization maps a high-precision value to one of a finite set of representable values.

For a uniform quantizer with step size Δ, the quantization error is:

```
e_q = x_hat - x
```

For rounding to nearest, the error is normally bounded by:

$$-\Delta/2 \leq e_q \leq \Delta/2$$

For truncation, the error is generally biased and its bound depends on the signed truncation convention.

If the quantization error is modelled as uniformly distributed and uncorrelated with the signal, its variance is:

$$\sigma_q^2 = \Delta^2 / 12$$

This is a useful engineering approximation, not a universal guarantee. Quantization error can be signal-correlated, especially for low-level periodic inputs and recursive filters.

Example 2 — Q1.15 quantization error

Let x = 0.3 and use Q1.15.

```
X = round(0.3 × 32768) = 9830

x_hat = 9830 / 32768 = 0.29998779297

e_q = x_hat - x = -0.00001220703
```

The Q1.15 LSB is 0.0000305176, so the magnitude of this rounding error is less than half an LSB.

4. Quantization noise and SQNR

For a full-scale sinusoid applied to an ideal uniform quantizer, the approximate signal-to-quantization-noise ratio is:

```
SQNR ≈ 6.02B + 1.76 dB
```

where B is the number of quantizer bits.

For a 12-bit ideal quantizer:

```
SQNR ≈ 6.02(12) + 1.76 = 74.00 dB
```

For a 16-bit ideal quantizer:

```
SQNR ≈ 6.02(16) + 1.76 = 98.08 dB
```

The formula assumes an ideal quantizer and a full-scale sinusoid. Actual performance can be worse because of analogue noise, clock jitter, nonlinearity, imperfect scaling and signal statistics.

5. Finite-word-length effects in digital filters

Finite precision can affect a filter at several points:

1. **Input quantization:** ADC samples are represented with finite resolution.
2. **Coefficient quantization:** Ideal filter coefficients are rounded to the available word length.
3. **Product quantization:** Multiplication results may be rounded or truncated.
4. **Accumulator quantization:** Sums may lose precision if the accumulator is too narrow.
5. **Output quantization:** The final result is converted to the output word length.

Coefficient quantization changes the implemented transfer function. In IIR filters, small coefficient changes can move poles and potentially degrade stability or response. For this reason, high-order IIR filters are commonly implemented as cascaded second-order sections (SOS) with carefully selected scaling.

6. Overflow: wrap-around versus saturation

An overflow occurs when the exact arithmetic result is outside the representable range.

Wrap-around arithmetic discards the excess high-order bits. A positive value can suddenly become negative in two's-complement arithmetic. This is usually undesirable in signal paths.

Saturation arithmetic clamps the result to the maximum or minimum representable value. Saturation avoids catastrophic sign reversal, but it introduces nonlinear distortion.

Example using signed 8-bit integers:

- representable range: -128 to +127;
- exact result: 120 + 20 = 140;
- wrap-around result: -116;
- saturated result: +127.

The arithmetic mode must be known for the target processor or DSP library. Do not assume all languages and hardware use the same overflow behaviour.

7. Accumulator sizing and guard bits

A filter often accumulates several products. Even when each input and coefficient is within range, the sum can exceed the range of the product format.

For an FIR filter:

$$y[n] = \Sigma(k=0 \text{ to } M)\ h[k]\ x[n-k]$$

If $|x[n]| \leq Xmax$, a conservative bound is:

$$|y[n]| \leq Xmax \times \Sigma\ |h[k]|$$

This bound helps determine the number of integer bits or the required input scaling.

Example 3 — Estimate the output headroom

Consider a four-tap FIR filter:

$$h = [0.25, 0.25, 0.25, 0.25]$$

and $|x[n]| \leq 1$.

Then:

$$\Sigma|h[k]| = 1$$

so:

$$|y[n]| \leq 1$$

The filter does not require extra peak headroom under this ideal bound.

Now consider:

$$h = [0.8, 0.8, -0.6, 0.4]$$

Then:

$$\Sigma|h[k]| = 2.6$$

and the conservative output bound is 2.6. A normalized input alone does not guarantee a normalized output. The implementation needs scaling, extra integer bits or saturation protection.

8. Quantization effects in IIR filters

An IIR filter uses feedback:

```
y[n] = Σ b[k]x[n-k] - Σ a[k]y[n-k]
```

Quantization inside the feedback loop can repeatedly feed rounding errors back into the system. Possible effects include:

- shifted pole locations and changed frequency response;
- increased noise at the output;
- overflow in internal states;
- zero-input limit cycles in some fixed-point realizations.

A limit cycle is a persistent non-zero output that can remain after the input becomes zero. It is associated with finite-precision nonlinearities in recursive structures.

Practical mitigation

- use second-order sections instead of one high-order polynomial;
- use a sufficiently wide accumulator;
- apply section scaling to control internal peaks;
- choose rounding and saturation deliberately;
- test zero-input response and low-level signals;
- compare floating-point and fixed-point outputs.

9. Real-time DSP constraints

A real-time DSP system must complete the processing for each sample or block before its deadline.

For sample rate `f_s`, the sample period is:

```
T_s = 1 / f_s
```

At 48 kHz:

```
T_s = 1 / 48000 = 20.833 µs
```

The processing budget per sample is therefore 20.833 microseconds in a sample-by-sample system. Interrupt overhead, memory access, communication and other tasks also consume time.

For a block of N samples:

```
T_block = N / f_s
```

For N = 128 at 48 kHz:

```
T_block = 128 / 48000 = 2.667 ms
```

Block processing can improve throughput, but it adds buffering latency. The acceptable block size depends on the application.

Example 4 — Estimate the computation budget

A processor runs at 200 MHz and the sample rate is 48 kHz.

Clock cycles available per sample:

```
200,000,000 / 48,000 = 4166.67 cycles/sample
```

If a filter requires 80 multiply-accumulate operations per sample, the arithmetic work may fit comfortably in this budget on a suitable processor. But the final decision must include instruction cost, memory stalls, interrupts, compiler output and other tasks. MAC count alone is not a timing proof.

10. Industry case study: power-quality harmonic measurement

A power-quality instrument samples voltage and current, estimates the fundamental component and measures harmonic content.

A simplified signal chain is:

```
Voltage/current sensor → anti-aliasing filter → ADC → digital scaling → windowing → FFT → harmonic magnitude calculation → reporting
```

For a 50 Hz system, a 10-cycle observation window has duration:

```
T = 10 / 50 = 0.2 s
```

If sampled at 5 kHz, the number of samples is:

```
N = 5000 × 0.2 = 1000 samples
```

A practical instrument must consider:

- ADC resolution and sensor dynamic range;
- anti-aliasing filter performance;
- fixed-point scaling through the FFT stages;
- overflow protection in butterfly operations;
- window coefficient precision;
- frequency deviation from nominal 50 Hz;
- latency and reporting update rate.

Engineering note: A 1000-point DFT is not radix-2. A production implementation may select a supported FFT length, use mixed-radix processing, or resample/window appropriately. The transform length must be chosen deliberately rather than assuming every FFT has a power-of-two size.

11. Embedded implementation workflow

A disciplined implementation process is:

1. Design and verify the filter in floating point.
2. Choose sample format and coefficient format.
3. Quantize coefficients and inspect the resulting frequency response.

4. Derive worst-case internal signal bounds.
5. Select accumulator width and scaling.
6. Choose rounding, saturation and overflow behaviour.
7. Implement using target-specific DSP instructions or a vetted DSP library.
8. Measure execution time on the actual hardware.
9. Test impulses, steps, full-scale sinusoids, multi-tone signals and zero-input conditions.
10. Compare fixed-point output against a floating-point reference.

Useful implementation ecosystems include ARM CMSIS-DSP for Cortex-M processors, vendor DSP libraries, FPGA fixed-point IP and carefully validated custom kernels.

12. Worked design exercise

A three-tap moving-average filter is:

```
y[n] = (x[n] + x[n-1] + x[n-2]) / 3
```

Assume the input is stored in signed Q1.15.

Tasks

1. Explain why the sum of three Q1.15 samples needs more integer headroom than one input sample.
2. Propose an accumulator format that avoids overflow for all inputs in [-1, 1).
3. Explain where division by three should occur.
4. Compare rounding and truncation at the output.
5. Estimate the group delay of the three-tap linear-phase FIR filter.
6. Explain whether the filter is appropriate for detecting fast transients in a protection relay.

Guidance

The sum can approach 3 in magnitude, so a raw 16-bit Q1.15 accumulator is insufficient. Use a wider accumulator or scale the sum before narrowing. The linear-phase three-tap moving average has group delay `(3-1)/2 = 1` sample. Smoothing can suppress noise, but it can also delay or attenuate fast transients; suitability depends on the protection function and its response-time requirements.

13. Review questions

1. What is the Q1.15 numerical range and resolution?
2. Why can coefficient quantization alter an IIR filter's stability?
3. Differentiate wrap-around and saturation arithmetic.
4. Why are guard bits used in an accumulator?
5. What is a zero-input limit cycle?
6. Calculate the sample period for a 20 kHz DSP system.
7. Why can block processing increase latency?
8. Why should high-order IIR filters often be implemented as second-order sections?
9. Why is a floating-point simulation insufficient to validate a fixed-point implementation?
10. List the tests required before deploying a real-time DSP algorithm.

14. Key takeaways

- Ideal DSP equations must be translated into finite-precision arithmetic.
- Word length, scaling and accumulator design are core engineering decisions.
- Saturation prevents wrap-around but does not eliminate distortion.
- Recursive filters are particularly sensitive to finite-word-length effects.
- Real-time correctness includes both numerical accuracy and meeting deadlines.
- Verification must use the actual target hardware and representative worst-case signals.

Suggested references

1. A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, Pearson.
2. J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, Pearson.
3. ARM, *CMSIS-DSP Software Library* documentation — fixed-point and floating-point DSP functions.
4. Texas Instruments, *Digital Signal Processing Applications with the TMS320 Family* — implementation-oriented DSP material.

End of Lecture 31